BỘ KHOA HỌC VÀ CÔNG NGHỆ
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

BÁO CÁO ĐỒ ÁN MÔN HỌC

XÂY DỰNG HỆ THỐNG NHÚNG HỖ TRỢ BÁN HÀNG THÔNG MINH

Môn học Xây dựng các hệ thống nhúng (INT1461)
Giảng viên hướng dẫn: Ths. Nguyễn Trọng Kiên
Thực hiện bởi nhóm 10:

Hồ Công Toản	**N22DCCN086**
Nguyễn Tấn Khoa	**N22DCCN042**
Đậu Văng Đăng Khoa	**N22DCCN040**
Hoàng Phi Long	**N22DCCN048**

HÀ NỘI, THÁNG 3 NĂM 2026

Lời cam đoan

Chúng tôi cam đoan rằng đồ án này là công trình nghiên cứu của nhóm chúng tôi dưới sự hướng dẫn của giảng viên. Các nội dung nghiên cứu và kết quả trong đồ án này là trung thực và chưa từng được công bố dưới bất kỳ hình thức nào. Những số liệu phục vụ cho việc phân tích, đánh giá được thu thập từ các nguồn khác nhau có ghi rõ trong phần tài liệu tham khảo.

Lời cảm ơn

Chúng tôi xin gửi lời cảm ơn chân thành đến Ths. Nguyễn Trọng Kiên đã tận tình hướng dẫn và hỗ trợ nhóm trong suốt quá trình thực hiện đồ án.

Chúng tôi cũng xin cảm ơn Học viện Công nghệ Bưu chính Viễn thông đã tạo điều kiện thuận lợi để nhóm hoàn thành đồ án này.

Tóm tắt

Dự án ShopNexus xây dựng hệ thống nhúng hỗ trợ bán hàng thông minh, bao gồm thiết bị ESP32 thu âm thanh và hình ảnh, máy chủ xử lý AI (khử nhiễu DTLN, nhận dạng giọng nói qua OpenAI Whisper, phân loại sản phẩm bằng XLM-RoBERTa), hệ thống backend Go với kiến trúc modular monolith, tìm kiếm hybrid vector qua Milvus, và giao diện web Next.js 16. Báo cáo này trình bày tiến độ thực hiện tính đến buổi 7.

MỤC LỤC

Lời cam đoan **i**

Lời cảm ơn **ii**

Tóm tắt **iii**

Mục lục **iv**

Danh sách bảng **vi**

Danh sách hình **vii**

1 Tổng quan dự án **1**

1.1 Giới thiệu 1

1.2 Kiến trúc tổng thể 1

1.3 Luồng xử lý chính 1

2 Thiết bị nhúng ESP32 **3**

2.1 Phần cứng 3

2.1.1 Cấu hình chân kết nối 3

2.2 Kiến trúc Dual-Core 3

2.3 Pipeline thu âm thanh 4

2.3.1 Cấu hình I2S 4

2.3.2 Chuẩn hóa tín hiệu 4

2.4 Pipeline streaming video 4

2.5 Giao thức điều khiển camera 5

2.6 Server trung gian (Node.js) 5

3 Các dịch vụ AI **6**

3.1 Dịch vụ khử nhiễu – Noise Reduction Service 6

3.1.1 Tổng quan 6

3.1.2 Mô hình DTLN 6

3.1.3 Kiến trúc hai giai đoạn 6

3.1.4 Hàm mất mát Negative SNR 8

3.1.5 Huấn luyện trên VIVOS 8

3.1.6 Pipeline xử lý 8

3.2 Dịch vụ nhận dạng giọng nói – Voice Rephraze Service 8
3.2.1 Tổng quan 8
3.2.2 Speech-to-Text 9
3.2.3 Sinh mô tả sản phẩm 9
3.2.4 Lưu trữ và quản lý 9
3.3 Dịch vụ phân loại sản phẩm – Product Classification Service 9
3.3.1 Tổng quan 9
3.3.2 Mô hình XLM-RoBERTa 9
3.3.3 API Endpoints 10
3.3.4 32 danh mục sản phẩm 10
3.3.5 Hiệu năng 10
3.3.6 Pipeline suy luận 11

4 Đánh giá và kết luận 12

4.1 Kết quả đạt được 12
4.2 Ưu điểm 12
4.3 Hạn chế 13
4.4 Kết luận 13
4.5 Hướng phát triển 13

Tài liệu tham khảo 14

DANH SÁCH BẢNG

1.1 Tổng quan các thành phần hệ thống 2

2.1 Thông số phần cứng ESP32 3
2.2 Phân công xử lý dual-core trên ESP32 4
2.3 Cấu hình I2S cho microphone INMP441 4
2.4 Các lệnh điều khiển camera chính 5

3.1 Các thông số chính của mô hình DTLN 7
3.2 Cấu trúc Separation Kernel – Giai đoạn 1 7
3.3 So sánh hai giai đoạn xử lý DTLN 7
3.4 Cấu hình huấn luyện DTLN trên VIVOS 8
3.5 Các phong cách viết mô tả sản phẩm 9
3.6 API endpoints dịch vụ phân loại 10
3.7 Benchmark hiệu năng phân loại sản phẩm 10

4.1 Tổng hợp kết quả các thành phần 12

DANH SÁCH HÌNH

CHƯƠNG 1

TỔNG QUAN DỰ ÁN

1.1 Giới thiệu

Dự án ShopNexus xây dựng hệ thống nhúng hỗ trợ bán hàng thông minh, cho phép người bán mô tả sản phẩm bằng giọng nói thông qua thiết bị ESP32-CAM. Hệ thống tự động khử nhiễu âm thanh, nhận dạng giọng nói, sinh mô tả sản phẩm và phân loại vào danh mục phù hợp.

1.2 Kiến trúc tổng thể

Hệ thống bao gồm 4 thành phần chính:

1. **Thiết bị nhúng ESP32-CAM:** Thu âm thanh (INMP441) và hình ảnh (OV5640), truyền dữ liệu real-time đến server qua TCP.
2. **Dịch vụ khử nhiễu (Noise Reduction):** Mô hình DTLN xử lý khử nhiễu giọng nói, triển khai trên Flask.
3. **Dịch vụ nhận dạng giọng nói (Voice Rephraze):** Chuyển đổi giọng nói thành văn bản và sinh mô tả sản phẩm, triển khai trên FastAPI.
4. **Dịch vụ phân loại sản phẩm (Product Classification):** Phân loại sản phẩm vào 32 danh mục sử dụng XLM-RoBERTa, triển khai trên FastAPI.

1.3 Luồng xử lý chính

1. ESP32 thu âm thanh từ microphone INMP441 (16-bit PCM, 16 kHz, mono) và gửi qua TCP port 3002 đến server trung gian.
2. Server trung gian (Node.js) nhận stream PCM, đóng gói thành file WAV và gửi đến **Noise Reduction Service** (port 5000).
3. Audio đã khử nhiễu được gửi đến **Voice Rephraze Service** (port 5001) để chuyển thành text và sinh mô tả sản phẩm theo phong cách viết được chọn.
4. Text mô tả được gửi đến **Product Classification Service** (port 5002) để phân loại vào 1 trong 32 danh mục thương mại điện tử.

5. Kết quả hiển thị trên giao diện web: spectrogram trước/sau khử nhiễu, text nhận dạng, mô tả đã sinh, và top-K danh mục dự đoán.

Bảng 1.1: Tổng quan các thành phần hệ thống

Thành phần	Công nghệ	Port	Vai trò
ESP32-CAM	Arduino/C++	TCP 3001, 3002	Thu âm thanh + hình ảnh
Server trung gian	Node.js + Express	3000	Relay TCP↔WebSocket, proxy API
Noise Reduction	Flask + TensorFlow	5000	Khử nhiễu DTLN
Voice Rephraze	FastAPI + OpenAI	5001	STT + sinh mô tả
Product Classification	FastAPI + PyTorch	5002	Phân loại 32 danh mục

CHƯƠNG 2

THIẾT BỊ NHÚNG ESP32

2.1 Phần cứng

Thiết bị sử dụng board ESP32-CAM tích hợp camera OV5640 và microphone INMP441 kết nối qua giao thức I2S.

Bảng 2.1: Thông số phần cứng ESP32

Thành phần	Thông số
Board	ESP32-CAM với PSRAM
Camera	OV5640, JPEG, QVGA (320×240) mặc định, hỗ trợ đến UXGA (1600×1200)
Microphone	INMP441 (I2S digital), 16 kHz, mono
Kết nối	WiFi 802.11 b/g/n
Giao thức	TCP (camera port 3001, audio port 3002)

2.1.1 Cấu hình chân kết nối

Camera OV5640 sử dụng bus dữ liệu 8-bit song song: D0–D7 (GPIO 5, 18, 19, 21, 36, 39, 34, 35), XCLK (GPIO 0), PCLK (GPIO 22), VSYNC (GPIO 25), HREF (GPIO 23), SDA (GPIO 26), SCL (GPIO 27), PWDN (GPIO 32).

Microphone INMP441 kết nối qua I2S: WS – Word Select (GPIO 2), SCK – Serial Clock (GPIO 14), SD – Serial Data (GPIO 15), sử dụng I2S port 1.

2.2 Kiến trúc Dual-Core

ESP32 có 2 nhân xử lý, được phân công như sau:

Việc tách riêng hai core đảm bảo luồng audio chạy liên tục không bị gián đoạn bởi camera capture, tránh hiện tượng buffer underrun.

Bảng 2.2: Phân công xử lý dual-core trên ESP32

Core	Nhiệm vụ	Chi tiết
Core 0	Audio Capture Task	Đọc I2S liên tục từ INMP441, chuyển đổi 32-bit→16-bit PCM, stream qua TCP port 3002. Stack: 16 KB.
Core 1	Camera + Main Loop	Chụp JPEG từ OV5640, gửi qua TCP port 3001, xử lý lệnh điều khiển camera từ server.

2.3 Pipeline thu âm thanh

2.3.1 Cấu hình I2S

Bảng 2.3: Cấu hình I2S cho microphone INMP441

Tham số	Giá trị
Chế độ	I2S_MODE_MASTER | I2S_MODE_RX
Tần số lấy mẫu	16.000 Hz
Bit depth đầu vào	32-bit (I2S native)
Bit depth đầu ra	16-bit PCM (sau chuyển đổi)
Kênh	Mono (chỉ kênh trái)
Số DMA buffer	16 (tăng từ 8 để ổn định)
Kích thước buffer	1024 mẫu (64 ms mỗi buffer)
Throughput	32 KB/s ($\approx$ 256 kbps)

2.3.2 Chuẩn hóa tín hiệu

Microphone INMP441 có độ phân giải 18-bit, đóng gói trong word 32-bit với dữ liệu nằm ở các bit cao. Quá trình chuẩn hóa:

1. Đọc mẫu 32-bit raw từ I2S DMA buffer.
2. Dịch phải 14 bit: $\mathrm{val} = \mathrm{raw_samples[i]} \gg 14$.
3. Cắt ngưỡng vào phạm vi 16-bit signed: $[-32768, 32767]$.
4. Lưu dưới dạng $\mathrm{int16_t}$ để truyền qua TCP.

2.4 Pipeline streaming video

Camera OV5640 được cấu hình chụp JPEG với double buffering (2 frame buffer). XCLK mặc định 12 MHz, chất lượng JPEG có thể điều chỉnh (10–63).

Mỗi frame được truyền qua TCP port 3001 với giao thức: [Length (4 bytes LE)][JPEG Data (variable)]. Buffer tĩnh 250 KB được cấp phát một lần trên PSRAM (hoặc heap) để tránh phân mảnh bộ nhớ.

2.5 Giao thức điều khiển camera

Trình duyệt gửi lệnh điều khiển qua WebSocket đến server trung gian, server chuyển tiếp qua TCP đến ESP32 với format: [0xA5][ID (1 byte)][Value (1 byte)].

Bảng 2.4: Các lệnh điều khiển camera chính

ID	Tham số	Phạm vi
1	JPEG Quality	10–63 (thấp = chất lượng cao hơn)
2	Frame Size	QQVGA đến UXGA (14 mức)
3–5	Brightness, Contrast, Saturation	-2 đến $+2$
6–9	AWB, AEC, AGC, AEC2	0/1 (bật/tắt)
16–17	H-Mirror, V-Flip	0/1
20	XCLK Frequency	6/8/12/16/24 MHz
21	Frame Buffer Count	1–4

ESP32 phản hồi cài đặt hiện tại qua packet với magic header 0xBEEF theo format: [0xBE][0xEF][ID][Va
cho tất cả 21 tham số.

2.6 Server trung gian (Node.js)

Server trung gian chạy trên Bun/Express đóng vai trò cầu nối giữa ESP32 và trình duyệt:

- **TCP Camera Server (port 3001):** Nhận JPEG frame từ ESP32, parse 4-byte length header, broadcast qua WebSocket /camera đến browser.
- **TCP Audio Server (port 3002):** Nhận stream 16-bit PCM từ ESP32, duy trì byte-alignment, broadcast qua WebSocket /audio đến browser.
- **HTTP API Proxy:** Chuyển tiếp request từ browser đến các AI services:
  - POST /api/process-voice → Denoise Service → STT Service
  - POST /api/gen-description → Voice Rephraze Service
  - POST /api/classify → Classification Service

Giao diện web hiển thị video streaming real-time, biểu đồ amplitude âm thanh (30 bars), spectrogram trước/sau khử nhiễu (WaveSurfer.js), kết quả nhận dạng và phân loại sản phẩm.

CHƯƠNG 3

CÁC DỊCH VỤ AI

3.1 Dịch vụ khử nhiễu – Noise Reduction Service

3.1.1 Tổng quan

Dịch vụ nhận file WAV chứa giọng nói lẫn nhiễu từ ESP32 và trả về audio đã khử nhiễu. Triển khai trên Flask 2.3+ với TensorFlow 2.13+, kiến trúc layered (API → Service → Model).

- **API:** POST /denoise – multipart/form-data, file WAV tối đa 50 MB.
- **Health check:** GET /health – trạng thái model và thông số.
- **Singleton Pattern:** Model được load một lần khi khởi động, cache trong bộ nhớ.

3.1.2 Mô hình DTLN

DTLN (Dual-signal Transformation LSTM Network) là mô hình khử nhiễu giọng nói được đề xuất bởi Westhausen và Meyer (2020), thiết kế cho xử lý thời gian thực trên thiết bị tài nguyên hạn chế.

3.1.3 Kiến trúc hai giai đoạn

Giai đoạn 1 – Miền tần số (STFT Domain): Tín hiệu đầu vào được chia thành các khung 512 mẫu, áp dụng STFT để thu được 257 bin tần số. Phổ biên độ $|X(f,t)|$ được chuẩn hóa qua InstantLayerNormalization, xử lý qua 2 lớp LSTM (128 units) với Dropout 0,25, rồi qua Dense + Sigmoid để tạo mask $M_1 \in [0,1]$. Mask được nhân với phổ biên độ gốc, kết hợp lại với phổ pha và biến đổi ngược (iFFT + Overlap-and-Add) để thu được tín hiệu $y_1(t)$.

Giai đoạn 2 – Miền đặc trưng (Feature Domain): Tín hiệu $y_1(t)$ được chiếu sang không gian 256 chiều qua Conv1D point-wise (kernel=1, stride=1). Sau InstantLayerNormalization và Separation Kernel thứ hai (cùng cấu trúc LSTM), Dense + Sigmoid tạo mask M_2 trong không gian đặc trưng. Giải mã qua Conv1D point-wise (256→512) với causal padding, rồi Overlap-and-Add tạo tín hiệu sạch $\hat{y}(t)$.

Bảng 3.1: Các thông số chính của mô hình DTLN

Thông số	Giá trị
Tần số lấy mẫu	16.000 Hz, mono
Kích thước khung (blockLen)	512 mẫu (32 ms)
Bước dịch khung (block_shift)	128 mẫu (8 ms), overlap 75%
Số bin tần số sau STFT	257 (= blockLen / 2 + 1)
Số đơn vị LSTM mỗi lớp	128 units
Số lớp LSTM mỗi giai đoạn	2 lớp
Dropout	0,25
Số bộ lọc Encoder Conv1D	256
Hàm kích hoạt mask	Sigmoid [0, 1]
Hàm mất mát	Negative SNR
Tổng số tham số	$\approx$ 1,8 triệu
Kích thước file model	3,9 MB (DTLN_vivos_best.h5)
Độ trễ xử lý	$\approx$ 32 ms (real-time capable)

Bảng 3.2: Cấu trúc Separation Kernel – Giai đoạn 1

Thành phần	Thông số	Vai trò
LSTM lớp 1	128 units, return_seq=True	Đặc trưng tần số cục bộ
Dropout	0,25	Giảm overfitting
LSTM lớp 2	128 units, return_seq=True	Phụ thuộc thời gian dài hạn
Dense	257 đơn vị	Ánh xạ về không gian bin tần số
Sigmoid	Đầu ra [0, 1]	Tạo mask $M_1(f,t)$

Bảng 3.3: So sánh hai giai đoạn xử lý DTLN

Tiêu chí	STFT (GĐ 1)	Conv1D Encoder (GĐ 2)
Loại biến đổi	Cố định (fixed basis)	Học từ dữ liệu
Chiều đầu ra	257 bin tần số	256 đặc trưng ẩn
Ý nghĩa vật lý	Rõ ràng, diễn giải được	Trừu tượng
Vai trò	Loại bỏ nhiễu tổng thể	Tinh chỉnh, phục hồi chi tiết

3.1.4 Hàm mất mát Negative SNR

Mô hình được tối ưu hóa bằng hàm mất mát:

$$\text{Loss} = -10 \cdot \log_{10} \left(\frac{E[s^2]}{E[(s-\hat{s})^2 + \varepsilon]} \right) \tag{3.1}$$

trong đó s là tín hiệu sạch, $\hat{s}$ là tín hiệu ước lượng, $\varepsilon = 10^{-7}$.

3.1.5 Huấn luyện trên VIVOS

Bảng 3.4: Cấu hình huấn luyện DTLN trên VIVOS

Tham số	Giá trị	Giải thích
Dữ liệu sạch	VIVOS	Tiếng Việt, $>$ 1 GB
Dữ liệu nhiễu	Microsoft DNS	Kết hợp random SNR
Batch size	32	Cân bằng GPU/gradient
Độ dài đoạn	3 giây	Phù hợp VIVOS (3–5s)
Epoch tối đa	50	Kết hợp dừng sớm
Learning rate	10^{-3}	ReduceLROnPlateau
Early Stopping	10 epoch	Tránh overfitting
Optimizer	Adam, clipnorm=3.0	Tránh bùng nổ gradient

3.1.6 Pipeline xử lý

Luồng xử lý khi nhận request:

1. Đọc file WAV qua SoundFile, validate sample rate (16 kHz).
2. Convert stereo→mono (nếu cần), cast sang float32.
3. Thêm batch dimension, chạy $\text{model.predict}()$.
4. Clip output vào $[-1, 1]$, ghi file WAV đã khử nhiễu.
5. Trả file qua HTTP response, xóa file tạm.

3.2 Dịch vụ nhận dạng giọng nói – Voice Rephraze Service

3.2.1 Tổng quan

Dịch vụ chuyển đổi giọng nói thành văn bản và sinh mô tả sản phẩm theo phong cách viết tùy chọn. Triển khai trên FastAPI với AsyncOpenAI client.

3.2.2 Speech-to-Text

- **Mô hình:** OpenAI gpt-4o-transcribe (cloud API).
- **Định dạng hỗ trợ:** WAV, MP3, FLAC, OGG, WebM, M4A.
- **Ngôn ngữ:** Tiếng Việt (đa vùng miền).
- **API:** POST /stt – upload file audio, trả về text.
- **Lịch sử phát triển:** PhoWhisper-tiny (local) → whisper-1 (OpenAI) → gpt-4o-transcribe.

3.2.3 Sinh mô tả sản phẩm

- **Mô hình:** GPT-4o-mini (temperature 0.75, max 600 tokens).
- **API:** POST /gen – nhận product_description (text từ STT) và style (phong cách viết).
- **Prompt engineering:** System prompt hướng dẫn AI đóng vai copywriter thương mại điện tử, trích xuất thông tin sản phẩm từ text lộn xộn của STT, bỏ qua từ lấp, lặp lại, lỗi phát âm; viết 3–5 câu ngắn gọn theo phong cách yêu cầu.

Bảng 3.5: Các phong cách viết mô tả sản phẩm

Phong cách	Mô tả
Văn minh	Lịch sự, trang trọng, ngôn ngữ chuẩn mực
Chuyên nghiệp	Kỹ thuật, rõ ràng, tập trung thông số
Thân thiện	Ấm áp, gần gũi, dễ tiếp cận
Văn hóa người Tày	Yếu tố văn hóa dân tộc, hình ảnh đặc trưng

3.2.4 Lưu trữ và quản lý

Các phong cách viết được lưu trong SQLite (styles.db) với schema đơn giản (id, name, description). Người dùng có thể thêm phong cách mới qua POST /add-style.

3.3 Dịch vụ phân loại sản phẩm – Product Classification Service

3.3.1 Tổng quan

Dịch vụ phân loại text mô tả sản phẩm vào 32 danh mục thương mại điện tử. Triển khai trên FastAPI 0.115 với PyTorch 2.5.1 và HuggingFace Transformers 4.46.3.

3.3.2 Mô hình XLM-RoBERTa

- **Model:** Lezh1n/xlm-roberta-ecommerce-classifier (HuggingFace Hub).

- **Base:** XLM-RoBERTa-base (Facebook AI) – hỗ trợ đa ngôn ngữ.
- **Task:** Multi-class sequence classification (32 classes).
- **Kích thước:** $\approx$ 1,1 GB, max sequence 512 tokens.
- **Accuracy:** 90,14%. **F1-score:** 90,00%.

3.3.3 API Endpoints

Bảng 3.6: API endpoints dịch vụ phân loại

Endpoint	Chức năng
POST /classify	Phân loại 1 sản phẩm, trả top-K predictions
POST /classify/batch	Batch tối đa 100 items (batch_size=32)
GET /categories	Danh sách 32 danh mục
GET /model-info	Metadata và metrics mô hình
GET /health	Health check

3.3.4 32 danh mục sản phẩm

Electronics, Computers & Networking, Mobile Phones & Tablets, Fashion & Clothing, Shoes & Footwear, Bags & Luggage, Watches, Jewelry, Fashion Accessories, Beauty & Personal Care, Health & Wellness, Sports & Outdoors, Home Furniture, Home Decor & Lighting, Kitchen & Dining, Bedding & Bath, Large Appliances, Small Appliances, Grocery & Food, Baby & Maternity, Toys & Games, Books & Media, Stationery & Office Supplies, Pet Supplies, Automotive & Motorcycle, Tools & Hardware, Garden & Outdoor Living, Musical Instruments, Video Games & Gaming, Software & Digital Goods, Arts & Crafts, Industrial & Commercial.

3.3.5 Hiệu năng

Bảng 3.7: Benchmark hiệu năng phân loại sản phẩm

Scenario	Latency	Throughput
Single inference (CPU)	$\approx$ 45 ms	22 req/s
Batch 10 items (CPU)	$\approx$ 180 ms	55 items/s
Single inference (GPU)	$\approx$ 15 ms	66 req/s
Batch 100 items (GPU)	$\approx$ 800 ms	125 items/s

3.3.6 Pipeline suy luận

1. Client gửi text mô tả sản phẩm qua POST request.
2. Validate input bằng Pydantic schema.
3. Tokenize input (max 512 tokens) qua AutoTokenizer.
4. Forward pass sinh logits cho 32 danh mục.
5. Softmax $\rightarrow$ top-K predictions với category ID, name, display name, score.
6. Trả response kèm inference time.

Model được load khi khởi động qua FastAPI Lifespan Context Manager, tự động detect GPU (fallback CPU), set evaluation mode.

CHƯƠNG 4

ĐÁNH GIÁ VÀ KẾT LUẬN

4.1 Kết quả đạt được

Bảng 4.1: Tổng hợp kết quả các thành phần

Thành phần	Kết quả
ESP32-CAM	Dual-core: audio 16 kHz liên tục + video JPEG streaming, điều khiển camera real-time (21 tham số)
Khử nhiễu DTLN	1,8M tham số, 3.9 MB, 32 ms latency, huấn luyện trên VIVOS tiếng Việt
Voice Rephraze	STT qua gpt-4o-transcribe + sinh mô tả qua GPT-4o-mini, 4+ phong cách viết
Phân loại sản phẩm	Accuracy 90,14%, F1 90,00%, 32 danh mục, 45 ms/request (CPU)
Server trung gian	Relay TCP↔WebSocket, proxy API, spectrogram visualization

4.2 Ưu điểm

- **Pipeline end-to-end:** Từ thu âm ESP32 → khử nhiễu → nhận dạng giọng nói → sinh mô tả → phân loại sản phẩm, hoạt động liền mạch.
- **DTLN nhỏ gọn:** Chỉ 3,9 MB (1,8M tham số), xử lý real-time 32 ms, phù hợp thiết bị nhúng tài nguyên hạn chế.
- **Dual-domain denoising:** Kết hợp STFT (miền tần số) và Conv1D Encoder (miền đặc trưng học được) khử nhiễu toàn diện hơn phương pháp đơn miền.
- **Phân loại chính xác:** XLM-RoBERTa đạt 90,14% accuracy, hỗ trợ đa ngôn ngữ nhờ nền tảng cross-lingual.
- **Dual-core ESP32:** Tách riêng audio/video trên 2 core đảm bảo không gián đoạn stream, 16 DMA buffer cho audio ổn định.
- **Sinh mô tả linh hoạt:** Nhiều phong cách viết, prompt engineering xử lý tốt text lộn xộn từ STT.

4.3 Hạn chế

- **DTLN chỉ mono channel:** Chưa khai thác beamforming hay mảng microphone.
- **Phụ thuộc cloud cho STT:** Voice Rephraze dùng OpenAI API, yêu cầu internet và phát sinh chi phí.
- **DTLN nhạy cảm nhiễu phi tĩnh:** Hiệu năng giảm với nhiễu biến đổi nhanh hoặc âm nhạc phức tạp.
- **Pha không tối ưu:** Giai đoạn 1 tái sử dụng pha gốc, có thể gây artifact khi nhiễu nặng.
- **ESP32 WiFi dependency:** Thiết bị phụ thuộc kết nối WiFi ổn định, không có buffer offline.

4.4 Kết luận

Hệ thống ShopNexus đã xây dựng thành công pipeline xử lý AI hoàn chỉnh cho bài toán hỗ trợ bán hàng bằng giọng nói trên nền tảng nhúng. Thiết bị ESP32-CAM khai thác hiệu quả kiến trúc dual-core để thu đồng thời âm thanh và hình ảnh. Mô hình DTLN được huấn luyện trên dữ liệu tiếng Việt VIVOS cho kết quả khử nhiễu tốt với latency phù hợp real-time. Dịch vụ phân loại sản phẩm đạt accuracy 90,14% trên 32 danh mục, đáp ứng yêu cầu ứng dụng thực tế.

4.5 Hướng phát triển

- Triển khai PhoWhisper local để giảm phụ thuộc cloud API và chi phí.
- Bổ sung multi-microphone beamforming cho ESP32.
- Thêm windowing function trong STFT của DTLN để giảm spectral leakage.
- Tối ưu mô hình phân loại cho tiếng Việt (fine-tune thêm trên dữ liệu Việt).
- Triển khai DTLN inference trực tiếp trên Raspberry Pi 4 với TF Lite.

TÀI LIỆU THAM KHẢO

[1] Donald E. Knuth. Literate programming. *The Computer Journal*, 27(2):97–111, 1984.

[2] Donald E. Knuth. *The TEX Book*. Addison-Wesley Professional, 1986.

[3] Leslie Lamport. *LATEX: a Document Preparation System*. Addison Wesley, Massachusetts, 2 edition, 1994.

[4] Michael Lesk and Brian Kernighan. Computer typesetting of technical journals on UNIX. In *Proceedings of American Federation of Information Processing Societies: 1977 National Computer Conference*, pages 879–888, Dallas, Texas, 1977.

[5] Frank Mittelbach, Michel Gossens, Johannes Braams, David Carlisle, and Chris Rowley. *The LATEX Companion*. Addison-Wesley Professional, 2 edition, 2004.